

Terminalx

```
codio@cleverlogo-eventcricket:~/workspace$ cd learning-poetry
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ poetry build
Building learning-poetry (0.1.0)
- Building sdist
- Built learning-poetry-0.1.0.tar.gz
- Building wheel
- Built learning_poetry-0.1.0-py3-none-any.whl
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ ls dist
learning_poetry-0.1.0-py3-none-any.whl  learning-poetry-0.1.0.tar.gz
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$
```

Guidex

Collapse

Wheels

Building a Wheel

It should come as no surprise that Poetry streamlines the process of building a wheel. For starters, there is no need for a `startup.py` file. Everything needed for your wheel is contained in the `pyproject.toml` file. Though it is a good idea to verify that the metadata is properly filled out. First chang into the `learning-poetry` directory.

cd learning-poetry

Use the `build` command to create a wheel.

poetry build

That's it, that's all you have to do. Poetry creates both a source distribution and a build distribution in the `dist` directory. List the contents of this directory; you should see a `tar.gz` file (source distribution) and a `whl` file (build distribution).

ls dist

▼ Did you notice?

Poetry only produces a pure-Python wheel. You do not have the ability to create a universal or platform-specific wheel.

Try these variations:

- Create only a build distribution. First we need to delete the `dist` directory and its contents. Then use the `build` command and the `--format` flag to specify only a wheel.

rm -r dist && poetry build --format wheel

List the contents of `dist` again and you should only see the wheel file.

ls dist

- Create only a source distribution. First we need to delete the `dist` directory and its contents. Then use the `build` command and the `--format` flag to specify only a source distribution.

rm -r dist && poetry build --format sdist

List the contents of `dist` again and you should only see the source distribution.

ls dist

Publishing a Package

Poetry also lets you upload your package to PyPI. This is done with the `publish` command. This is much easier than using `pip`, which requires some intermediate steps like downloading and running `twine` before uploading to PyPI.

PyPI has the ability to display lots of useful information about your package. All of this can be specified in the `pyproject.toml` file and automatically added to the PyPI page. You should strongly consider providing information like:

- Homepage
- Documentation
- License
- Maintainers
- README
- Repository
- Keywords
- ...and many more

See the Poetry documentation for more information. Once your package is ready for distribution, use the `publish` command to send it to PyPI.

poetry publish

Reading Question

Fill in the blank with either True or False.

- ☐ False - Poetry requires a `setup.py` file when building a wheel.
- ☐ False - Poetry can only build a wheel file; it cannot build a source distribution.
- ☐ True - Poetry can publish packages to PyPI.

Here are the correct answers:

- False** - Poetry requires a `setup.py` file when building a wheel.
- False** - Poetry can only build a wheel file; it cannot build a source distribution.
- True** - Poetry can publish packages to PyPI.

Poetry uses the `pyproject.toml` file to help it build a wheel. There is no need a for a `setup.py` file. By default, Poetry creates a build and source distributions. You can specify to create only one of those option, but it can create a source distribution. Poetry publishes to PyPI by default.

Check It!

https://codio.com/sandipan-dey/poetry/terminal/backend-guide

1/1